

Pandas — Шпаргалка

Усі атрибути та методи, потрібні для 120 завдань практикуму. Усюди `df` — це ваш `DataFrame`, завантажений з CSV.

0 Підготовка

```
import pandas as pd
```

Імпорт бібліотеки зі скороченням **pd**.

```
df = pd.read_csv("file.csv")
```

Завантажити дані з CSV-файлу у **DataFrame**.

1 Перший погляд

завд. 1–15

```
df.head()    # або head(10)
df.tail()    # або tail(3)
```

Перші / останні рядки. У дужках — скільки рядків (за замовч. 5).

```
df.shape     # (рядки, стовпці)
df.shape[0]  # кількість рядків
df.shape[1]  # кількість стовпців
```

Розмір таблиці. Це атрибут — без дужок.

```
df.columns   # назви стовпців
df.dtypes    # типи даних
df.info()    # повний огляд
df.describe() # статистика чисел
```

Знайомство з таблицею: стовпці, типи, статистика.

```
len(df)      # кількість рядків
```

Функція **len()** теж повертає кількість рядків.



shape, columns, dtypes — атрибути (без дужок). **head(), info(), describe()** — методи (з дужками).

2 Вибір стовпців

завд. 16–30

```
df["city"] # один стовпець
```

Одинарні дужки → один стовпець (Series).

```
df[["city", "total"]] # кілька
```

Подвійні дужки → кілька стовпців (DataFrame).

```
df["genre"].head() # поєднання  
df[["city", "total"]].tail()
```

Методи можна **ланцюжком**: спочатку вибір, потім head/tail.



Один стовпець — одинарні дужки. Кілька стовпців — **подвійні** дужки [[]].

3

Агрегати по стовпцю

завд. 31–50

```
df["total"].sum() # сума  
df["total"].mean() # середнє  
df["total"].max() # максимум  
df["total"].min() # мінімум  
df["total"].count() # кількість
```

Числові агрегати — застосовуються до стовпця-числа.

```
df["city"].unique() # список значень  
df["city"].nunique() # їх кількість
```

unique() — унікальні значення, **nunique()** — їх число.

```
df["city"].value_counts()
```

value_counts() — скільки разів зустрічається кожне значення. Дуже корисна!

4

Фільтрація — одна умова

завд. 51–70

```
df[df["city"] == "Київ"]
```

Дорівнює — подвійний знак ==.

```
df[df["total"] > 500]  
df[df["total"] < 200]  
df[df["price"] >= 400]  
df[df["price"] <= 300]
```

Порівняння: >, <, >=, <=.



Принцип: усередині df[...] пишемо умову, що дає True/False для кожного рядка.

5

Фільтр + підрахунок

завд. 71–80

```
len(df[df["city"] == "Київ"])
```

Скільки рядків відповідає умові — обгорніть фільтр у `len()`.

```
df[df["city"]=="Львів"]["total"].sum()
```

Агрегат після фільтра: спочатку фільтруємо, потім беремо стовпець і агрегат.



Схема: `df[ умова ][ "стовпець" ].агрегат()` — фільтр, стовпець, агрегат.

6

Фільтр — кілька умов

завд. 81–95

```
df[(df["city"]=="Київ") &  
   (df["total"]>500)]
```

`&` означає **І** (обидві умови мають виконатись).

```
df[(df["genre"]=="Детектив") |  
   (df["genre"]=="Наука")]
```

`|` означає **АБО** (достатньо однієї умови).



Використовуйте `&` та `|`, а НЕ `and/or`.
Кожну умову беріть у **круглі дужки**!

7

Нові стовпці

завд. 96–105

```
df["full_price"] = df["unit_price"] * df["quantity"]
```

Новий стовець = вираз. Операція застосовується до всього стовпця одразу (векторизація).

```
df["is_big"] = df["total"] > 700
```

True/False стовець — результат умови.

```
df["total"].round()      # округлити  
df["city"].str.upper()   # ВЕЛИКІ літери
```

`round()` для чисел, `.str.upper()` для тексту.

8

Сортування

завд. 106–112

```
df.sort_values("total")
```

За **зростанням** (від найменшого).

```
df.sort_values("total", ascending=False)
```

`ascending=False` → за **спаданням** (від найбільшого).

```
df.sort_values("total", ascending=False).head(3)
```

Топ-N: відсортуйте за спаданням і візьміть `head(N)`.

```
df.groupby("city")["total"].sum()
```

Схема: `groupby`(за чим) [що рахувати] `.агрегат()`.

```
df.groupby("genre")["total"].mean()
df.groupby("manager")["order_id"].count()
```

Агрегат будь-який: **sum**, **mean**, **count**, **max**, **min**.

```
df.groupby("genre")["total"].sum() \
.sort_values(ascending=False)
```

Результат можна одразу **відсортувати**.

```
df.groupby("city")["total"] \
.agg(["mean", "min", "max"])
```

.agg([...]) — кілька агрегатів одразу.

```
df.groupby(["city", "genre"])["total"].sum()
```

Групування за **кількома стовпцями** — список у дужках.

```
.sum()      # сума
.mean()     # середнє
.max()      # максимум
.min()      # мінімум
.count()    # кількість непорожніх
.nunique()  # кількість унікальних
.round()    # округлення
```

Працюють і на **стовпці** (`df["x"].sum()`), і після **groupby**.